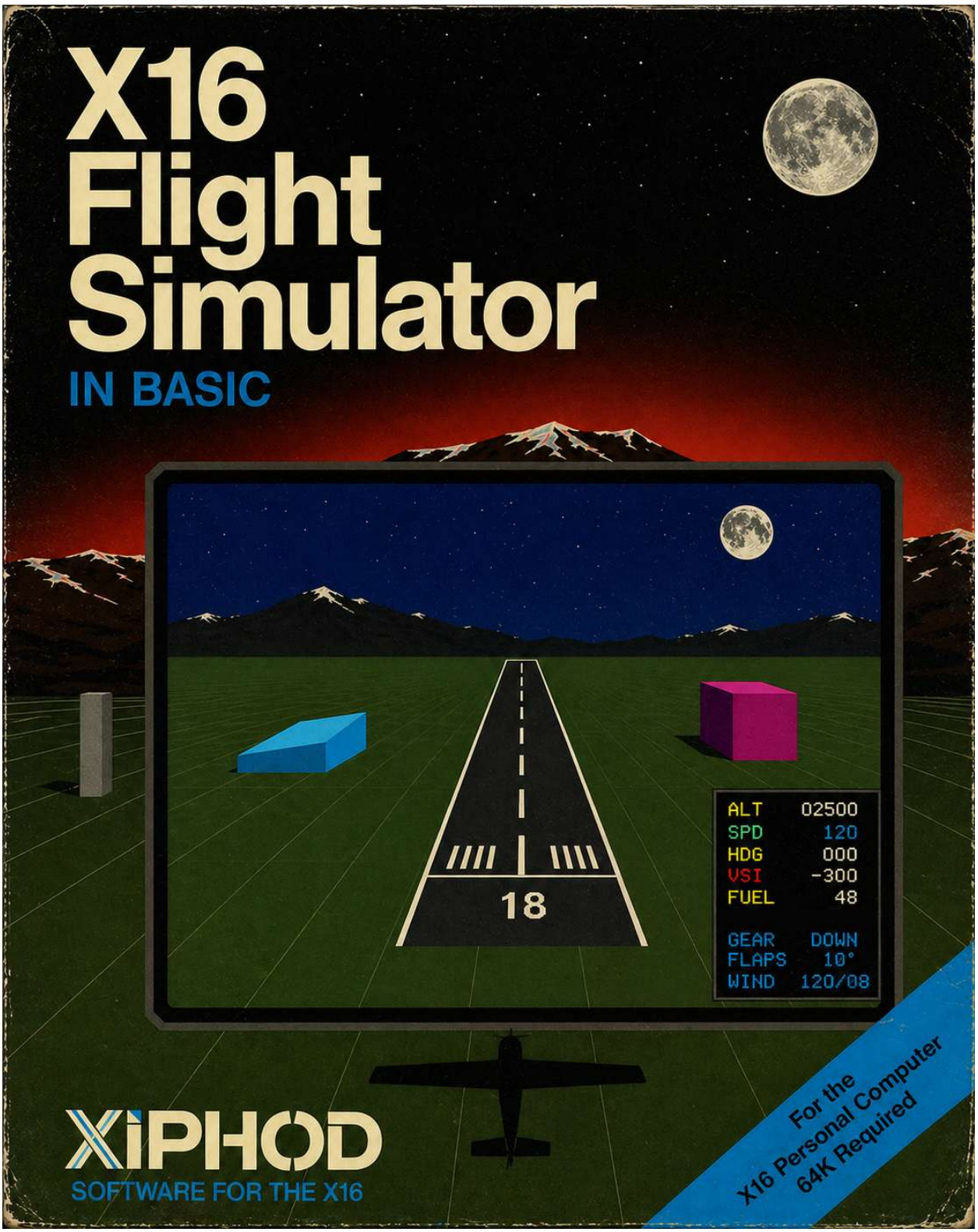
The X16 Flight Simulator V1.0



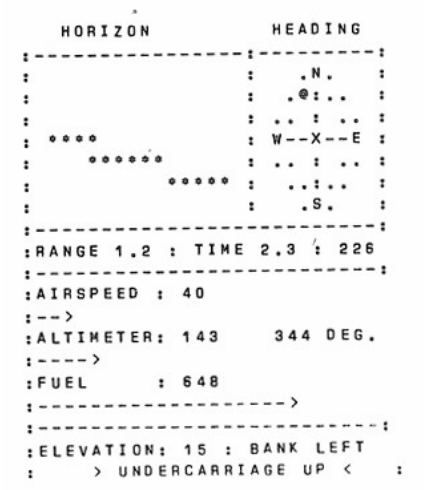
A development collaboration between Steve Lewis and ChatGPT. After 82 iterations across three six-hour sessions, the build is “feature complete” of all original intended goals.

Features

- An actual “3D” flight environment. You can circle-360 and observe interesting scenery.
- Two initial on-approach scenarios: south vs north
- Rudder, Thrust, Pitch, Roll, Flaps controls (and Landing Gear)
- Varied wind consideration: tailwind, headwind, crosswind
- Fuel constrained flight
- Recorded Flight Log to verify your ~~CRASH~~LANDING
- Windshield “virtual world” view and HUD gauge visualization
- Developed in BASLOAD, resulting tokenized BASIC is about 25KB

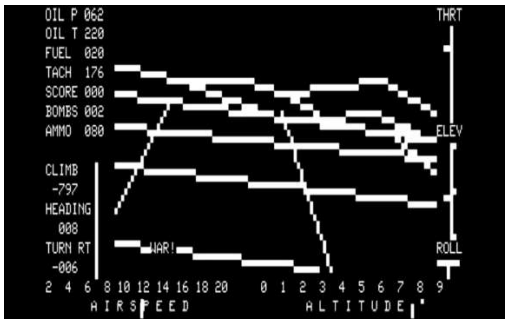
Forward

Many years ago (1988), I found a Commodore PET 4016 abandoned in a dumpster. At the time, I didn’t realize this was already ancient tech by then. But I was fascinating to find a fully working computer. Some time later, I came across Tim Hartnell’s “Creating Simulation Games On Your Computer” (1986). One of the BASIC examples therein was a simple Flight Simulator that depicted horizon changes (using asterisks) as you banked left/right.



I managed to get a version of Tim’s Flight Sim working on the Commodore PET. Weighing in at 250 lines, I found the “artificial horizon” fascinating. Meanwhile, on the Apple][and TRS-80 platforms, SubLogic had developed FS1 (1980) for those platforms. Both versions featured a top down (map) and “real-world” view, while the Apple][version also had actual gauge-looking HUD graphics.

BELOW: FS1 (1980) Apple][(map view) and TRS-80 (semi-graphic 3D world view)



Purely as a weekend experiment, I decided to see if modern-day AI could help me produce an interactive real-time Flight Simulator in BASIC. After an initial proof of concept panned out (affectionately called a “horizon simulator”), I decided to press on and make it a full featured experience.

I want to emphasize; this was not a simple linear “ask AI to do it” product. The AI “went bad” several times and started making worse results. Multiple course corrections and back-tracking was necessary. I did all the testing manually, sharing video playback and screenshots of results with the AI to clarify results. But full disclosure, by intent: This was “vibe coded,” meaning I was never hands on with the code itself.

There were numerous times I almost gave up on this, deeming BASIC even at 8MHz as inadequate to do this, or when the AI gradually became unresponsive as its context-capacity filled up. But I stuck with it and I consider this development a positive collaborative effort. Meaning, the end result was better than either of us would have accomplished on our own. I had to teach the AI a few things along the way (like how X16 BASIC doesn’t support ELSE, how FOR loop increment variables can’t use % modifier, and a few optimization tricks in BASIC pulled from my own experience), while the AI convinced me this was all possible without using any trig functions (and helped navigate an approach for multiple scenery objects, not just a simple horizon bar).

The result here is 1299 lines if plain text BASLOAD, that gets tokenized into 916 lines of X16 BASIC. And while still a little sluggish to play, the results here exceeded my expectations.

Loading and Starting the Game

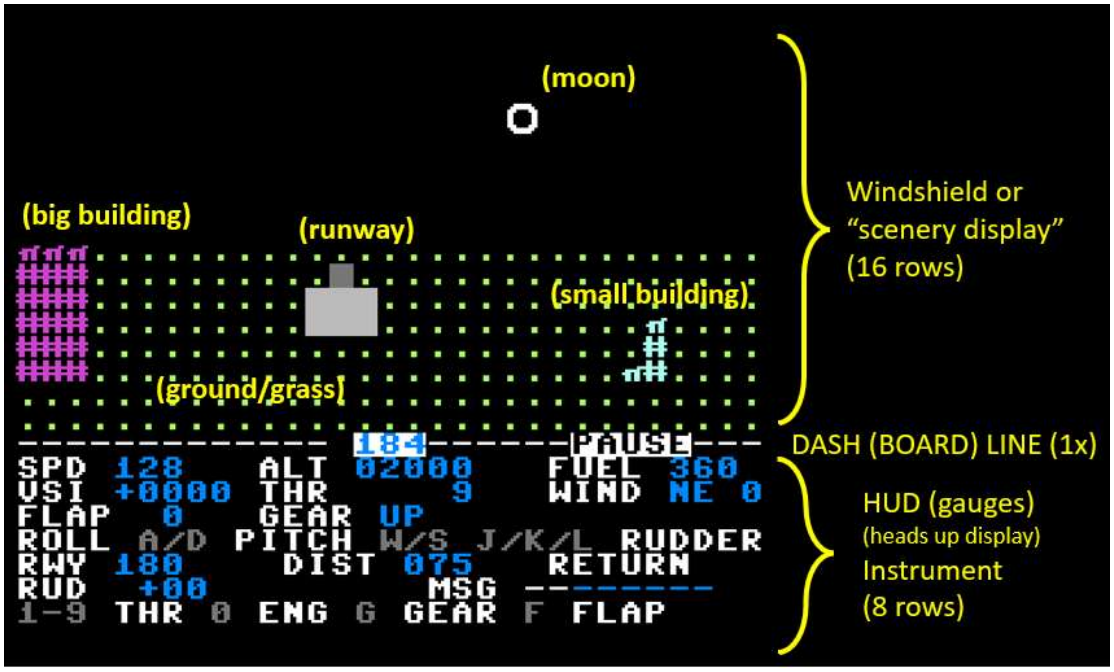
The source code is in BASLOAD, which the Commander X16 has the built in BASLOAD command used to "compile" the plain text code into tokenized BASIC.

FSIM16.BASL.TXT	X16 BASIC Source Code (plain text)
FSIM16.PRG	The actual runtime (tokenized X16 BASIC)

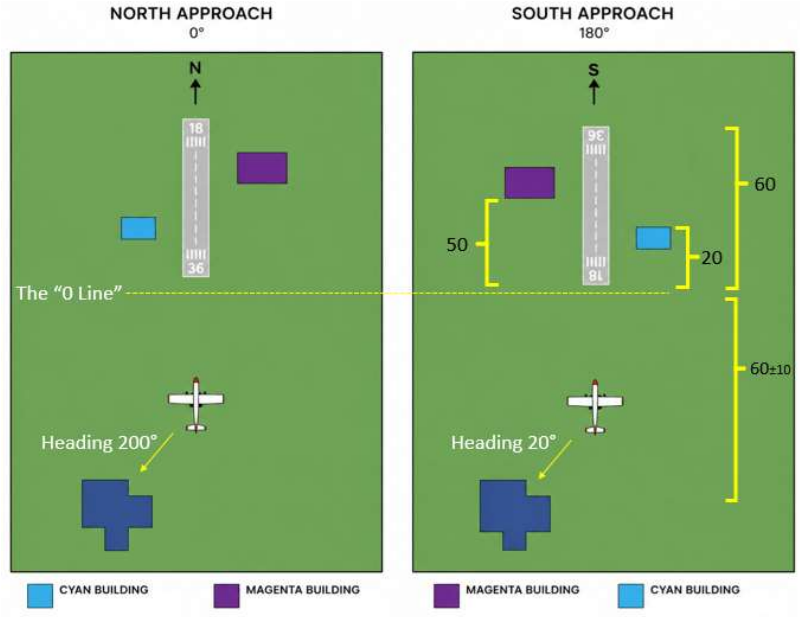
The simulation is a self-contained .PRG program, which the X16 LOAD command will automatically load into the appropriate address (using a header in the .PRG itself). Then issuing the RUN command will start the program.

The Display Layout and Game Objective

On startup, the simulation will randomly choose two approach options: (1) landing to the north (initial heading near 0 degrees and “magenta building” on the right) or (2) landing to the south (initial heading near 180 degrees and “magenta building” on the left; you will also see the moon above the horizon, since the scenario pretends the runway to be in the northern hemisphere).



The scenario can be depicted as follows, referencing the DIST (distance) metric: You start at a DIST of 60+/-10 from the runway (and altitude of 2000ft). Crossing onto the runway is when DIST is 0. The end of the runway is at DIST +60. The magenta building is always “further back” at DIST +50, with the cyan building at +20. Going down the runway, you can “pass” these buildings and they will disappear from view. If you bank around away from the runway, you can find a small lake behind you.



The simulation starts you on a landing approach, so the objective is to land successfully on that runway. You do have enough fuel to explore a little bit. The game approximately models a small Cessna 150, except that you do have retractable landing gear. The game does not permit barrel rolls or huge pitched loops. Being a trainer, for safety reasons, the roll and pitch are capped to about 50 degrees.

You start off at nearly full throttle, pacing you at about 120 knots. The main goal is:

- Align your heading with the runway
- Drop 2000 feet while reducing the airspeed to about 60 knots
- Achieve this on the runway itself (not too short or too far)
- (don't forget the landing gear!)

(1) Alignment: You start with a semi-random heading, basically +/-5 degrees of the intended landing approach. You can use RUDDER CONTROLS to steer closer to the perfect 360 or 180 degree heading (using J and L keys), or you can roll left/right slightly (A and D keys). Your heading is displayed at the center of the dash line.
(2) Reducing altitude: You can do this by enabling FLAPS (press F to cycle through 0, 1, 2 options), reducing throttle power (using numbers 1-9), or by pitching down a little (W and S keys). Landing takes time and requires patience! Hot rod landings often fail, but you're welcome to try. The instrument to watch is the VSI (vertical speed indicator) which is approximately calibrated to FEET PER MINUTE. Negative values mean you are losing altitude, positive indicate you are gaining altitude.
(3) Landing on runway: You start with NEGATIVE DIST, meaning you are short of the objective goal. You can still land early without a crash, but it will be counted as EARLY. Once DIST reaches 0, you are above the runway and can drop down to the final ALTITUDE of 0. But there is only so much concrete! You need to get wheels on the ground before DIST +60, or it counts as MISSED.

As mentioned, don't forget to lower the landing gear prior to landing (G key).

My Recommended Approach Sequence

- Rudder control to match heading 0 (if north) or 180 (is south); both buildings should come into view
- Throttle down from 9 to about 6 or 7 (!)
- Flaps to 2, do one or two pitch down (W key)
- Wait a little bit, when Altitude about 800-1000ft, then do G for gear down
- When DIST is at about -5, Altitude should be about 200-500ft and speed under 100.
- Probably pitch back up by this point, and throttle down to about 3 or 4
- When DIST crosses 0, adjust FLAPS as needed to get the VSI to around -300 to -500

A successful landing will show a message LANDED. Too hard of a landing will show CRASH message (and cracks appear on the windshield). It is possible to STALL, if you pitch up and reduce power. Another way to quickly STALL and CRASH is to just turn the engine off (0).

CONTROLS

```
PITCH DOWN          W
PITCH UP            S
ROLL LEFT           A
ROLL RIGHT          D
GEAR TOGGLE         G
ADJUST FLAPS        F   (0 is nominal, 1 is mild, 2 is max)
THRUST              1-9 (generally 1-2 never used, 3-4 landing, 9 max)
RUDDER LEFT         J
RUDDER CENTER       K
RUDDER RIGHT        L
TOGGLE ENGINE       0   (turn off engine to try some STALL tricks)
FUEL HACK           U   (disable fuel, fly forever!)
LOG GAME STATE      SPACE
RESTART/RESET       R   (only after LAND/CRASH or EARLY/MISS condition)
```

The Log System

The simulation maintains a log of the game to `FLIGHT.LOG` file. This includes the initial conditions of the scenario, then any input command you give records the navigation state at that time. You can also press SPACEBAR to log current state at any time. When you land (or crash), the log file is closed. If you want to save the contents, press X to exit the program and copy FLIGHT.LOG to another filename or folder (or if using the emulator, do so externally).

If you question a CRASH condition, consult the FLIGHT.LOG to examine what happened. The log could also technically be used to re-play a scenario. That feature isn't completed yet, but the intent is the log should have enough information preserved to do so. The log file is overwritten again if you restart the simulation.

Development Notes

Early concept art, that I drew manually using Excel. By intent, I don't use the 32nd column. By having an "odd number of columns" (31) we then have a definitive center line.

I had a plan for doing a DAY/NIGHT cycle option (with clouds in day and stars at night), but for now we just stuck with the night option. Since the performance is still a little sluggish, I didn't do the cloud or star scenery yet. But the moon is there, a couple buildings, and a small lake. And the heading is on the top of the dash.